

Week 2

Course Name : Problem solving ans Testing Using java

Course Code : 10211CS227

Name : Podaralla Nandini

VTU no : 28320

Task1

Given a **zero-based permutation** **nums** (**0-indexed**), build an array **ans** of the **same length** where $\text{ans}[i] = \text{nums}[\text{nums}[i]]$ for each $0 \leq i < \text{nums.length}$ and return it.

A **zero-based permutation** **nums** is an array of **distinct** integers from 0 to $\text{nums.length} - 1$ (**inclusive**).

Example 1:

Input: `nums = [0,2,1,5,3,4]`

Output: `[0,1,2,4,5,3]`

Explanation: The array **ans** is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]],  
nums[nums[3]], nums[nums[4]], nums[nums[5]]]  
     = [nums[0], nums[2], nums[1], nums[5], nums[3], nums[4]]  
     = [0,1,2,4,5,3]
```

Program:

```
class Solution {  
    public int[] buildArray(int[] nums) {  
        int n = nums.length;  
  
        for (int i = 0; i < n; i++) {  
            nums[i] = nums[i] + n * (nums[nums[i]] % n);  
        }  
        for (int i = 0; i < n; i++) {  
            nums[i] = nums[i] / n;  
        }  
  
        return nums;  
    }  
}
```

Task2

Given the array nums consisting of $2n$ elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$.

Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Example 1:

Input: nums = [2,5,1,3,4,7], n = 3

Output: [2,3,5,4,1,7]

Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is [2,3,5,4,1,7].

Program:

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
  
        int[] ans = new int[nums.length];  
  
        int j = 0;  
  
        for (int i = 0; i < n; i++) {  
            ans[j++] = nums[i];  
            ans[j++] = nums[i + n];  
        }  
  
        return ans;  
    }  
}
```

Task 3:

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length n where `gain[i]` is the net gain in altitude between points i and $i + 1$ for all $(0 \leq i < n)$. Return the highest altitude of a point.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

```
class Solution {
    public int largestAltitude(int[] gain) {
        int altitude = 0;
        int highest = 0;

        for (int i = 0; i < gain.length; i++) {
            altitude += gain[i];

            if (altitude > highest) {
                highest = altitude;
            }
        }

        return highest;
    }
}
```

Task 4:

In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail). Deque interfaces can be implemented using various types of collections such as LinkedList or ArrayDeque classes. For example, deque can be declared as:

```
Deque deque = new LinkedList<>();
```

or

```
Deque deque = new ArrayDeque<>();
```

You can find more details about Deque [here](#).

In this problem, you are given integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size .

Note: Time limit is second for this problem.

Input Format

The first line of input contains two integers N and : M representing the total number of integers and the size of the subarray, respectively. The next line contains Nspace separated integers.

Output Format

Print the maximum number of unique integers among all possible contiguous subarrays of M size .

Program:

```
import java.util.*;

public class test {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int m = sc.nextInt();
        Deque<Integer> deque = new ArrayDeque<>();
        HashMap<Integer, Integer> map = new HashMap<>();

        int max = 0;
```

```
for (int i = 0; i < n; i++) {  
    int num = sc.nextInt();  
  
    deque.addLast(num);  
    map.put(num, map.getOrDefault(num, 0) + 1);  
  
    // Window size becomes m  
    if (deque.size() == m) {  
        max = Math.max(max, map.size());  
  
        // Remove the first element before moving to next window  
        int removed = deque.removeFirst();  
  
        map.put(removed, map.get(removed) - 1);  
  
        if (map.get(removed) == 0) {  
            map.remove(removed);  
        }  
    }  
  
    System.out.println(max);  
}  
}
```

Task 5:

In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). `is` an example of a set, but `is` is not a set. Today you will learn how to use sets in java by solving this problem. You are given `pairs` of strings. Two pairs `and` are identical if `and` . That also implies `is` not same as `.` After taking each pair as input, you need to print number of unique pairs you currently have.

Input Format

In the first line, there will be an integer `T` denoting number of pairs. Each of the next `T` lines will contain two strings seperated by a single space.

Output Format

Print `T` lines. In the `ith` line, print number of unique pairs you have after taking `ith` pair as input.

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int t = s.nextInt();
        String [] pair_left = new String[t];
        String [] pair_right = new String[t];
```

```
for (int i = 0; i < t; i++) {  
    pair_left[i] = s.next();  
    pair_right[i] = s.next();  
}  
HashSet<String> set = new HashSet<>();
```

```
for (int i = 0; i < t; i++) {  
  
    String pair = pair_left[i] + " " + pair_right[i];  
  
    set.add(pair);  
  
    System.out.println(set.size());  
}  
}  
}
```